Snowflake CLI (`snow`) — Developer Cheatsheet

The `snow` CLI for Snowflake: deploy apps, run SQL, manage objects, and call Cortex AI — all from your terminal or CI/CD pipeline.

> [!IMPORTANT] Community cheatsheet — not official Snowflake documentation. For the authoritative reference, see Snowflake CLI docs.

Table of Contents

Install & Update

```
# run in terminal
pip install snowflake-cli                    # pip (recommended)
pipx install snowflake-cli                   # pipx (isolated)
brew install snowflake-cli                   # macOS Homebrew

snow --version                               # verify install
snow update                                  # update to latest
snow update 3.18.0                           # pin to specific
version
```

Quick Start By Role

If you are a...	Focus on
Data Engineer / Analyst	`snow sql`, `snow object`, `snow stage`
App Developer (Streamlit, Native Apps)	`snow streamlit`, `snow app`
ML / AI Engineer	`snow cortex`, `snow snowpark`
Platform / DevOps	`snow connection`, `snow dcm`, `snow spcs`

Connection

Connections are defined in `~/.snowflake/connections.toml` (or `$SNOWFLAKE_HOME/connections.toml`).

```
# run in terminal
snow connection add                              # interactive wizard
snow connection add \
  --connection-name myconn \
  --account orgname-accountname \
  --user myuser \
  --authenticator OAUTH_AUTHORIZATION_CODE \  # recommended (see
snowflake-gh-authn-cheatsheet)
  --client-store-temporary-credential true

snow connection list                             # list all
connections
snow connection set-default myconn           # set active default
snow connection test -c myconn               # test a named
connection
snow connection test -x                      # test via env vars
(no config needed)

# Generate a workload identity token (for CI/CD OIDC flows)
snow connection generate-workload-identity-token --workload-
identity-provider OIDC
```

> [!TIP] `-c` selects a named connection; `-x` (`--temporary-connection`) builds the connection from environment variables instead of `connections.toml` — ideal for CI/CD.

Cortex AI (`snow cortex`)

> [!NOTE] Available models vary by region. Check the supported models list for your account. Commonly available: `claude-3-5-sonnet`, `llama3.1-70b`, `mistral-large2`, `snowflake-arctic`. Use `--model` to specify; default is account-level.

Ask a question or run a prompt:

```
# run in terminal
snow cortex complete "Summarize the benefits of Snowflake
clustering"
snow cortex complete "Summarize the benefits of Snowflake
clustering" \
  --model claude-3-5-sonnet

# Multi-turn conversation from a JSON file
snow cortex complete --file samples/complete_datacloud.json
```

Extract an answer from a text passage:

```
snow cortex extract-answer \
  "What is the capital of France?" \
  "France is a country in Western Europe. Its capital city is
Paris."
```

Text analysis shortcuts:

```
snow cortex sentiment samples/sentiment.txt        # sentiment
score (-1 to 1)
snow cortex summarize samples/summarize.txt         # summarize a
document
snow cortex translate samples/translate.txt \
  --source-language French \
  --target-language English
```

SQL (`snow sql`)

```
# run in terminal
snow sql -q "SELECT current_version()"             # inline query
snow sql -f my_script.sql                          # run a file
cat my_script.sql | snow sql --stdin               # pipe from
stdin
snow sql -f script.sql -c prod                     # use a
specific connection

# Variable substitution (STANDARD syntax — recommended)
snow sql -q "GRANT USAGE ON DATABASE <% db %> TO ROLE <% role %>"
\
  -D "db=mydb" -D "role=analyst"
```

```
# Access environment variables (requires snowflake.yml env section
or --env)
snow sql -q "GRANT ROLE admin TO USER <% ctx.env.SNOWFLAKE_USER
%>" \
  --env SNOWFLAKE_USER=$SNOWFLAKE_USER

# Restrict to local files only (safe for untrusted SQL in CI)
snow sql -f script.sql --local-only

# Multiple files sequentially
snow sql -f setup.sql -f data.sql -f verify.sql

# Execute asynchronously (returns query ID immediately)
snow sql -q "INSERT INTO big_table SELECT * FROM source;"
```

Objects (`snow object`)

Manage most Snowflake objects via `snow object`:

```
# run in terminal
snow object create warehouse mywh \
  '{"warehouse_size": "X-SMALL", "auto_suspend": 300}'

snow object list warehouse                          # list
warehouses
snow object list table --like 'orders%' \
  --database mydb --schema public                   # filter by
name pattern
snow object list schema --in database mydb

snow object describe warehouse mywh                # full details
snow object drop table mydb.public.old_table       # drop an object
```

Supported types: `warehouse`, `database`, `schema`, `table`, `view`, `function`, `procedure`, `role`, `user`, `stage`, `task`, `stream`, `pipe`, and more. Run `snow object --help` for the full list.

Stages (`snow stage`)

```
# run in terminal
snow stage create mydb.public.my_stage            # create a stage

# Upload files
snow stage copy ./data/ @mydb.public.my_stage/data/           #
local → stage
snow stage copy @mydb.public.my_stage/data/ ./local_copy/     #
stage → local

snow stage list @mydb.public.my_stage              # list files
```

```
snow stage list @mydb.public.my_stage --like '*.sql'  # filter

# Execute SQL files directly from a stage
snow stage execute @mydb.public.my_stage/migration.sql

# Remove files
snow stage remove @mydb.public.my_stage/old_data.csv
snow stage remove @mydb.public.my_stage/archive/     # remove a
directory
```

Streamlit (`snow streamlit`)

```
# run in terminal — from project directory containing
snowflake.yml
snow streamlit deploy                                # deploy to
Snowflake
snow streamlit deploy --replace                      # replace
existing app

snow streamlit list                                  # list all
apps
snow streamlit describe my_app                       # app details
snow streamlit get-url my_app                        # open URL in
browser

# Stream live logs (SPCSv2 container runtime only; new in 3.18.0)
snow streamlit logs my_app --tail 100                # last 100
lines
snow streamlit logs my_app --tail 0 --follow         # follow live

snow streamlit drop my_app                           # remove app
```

Minimal `snowflake.yml` for Streamlit:

```yaml
definition_version: 2
entities:
  my_app:
    type: streamlit
    identifier: my_app
    stage: my_stage
    main_file: app.py
    query_warehouse: compute_wh
```

Apps (`snow app`)

Supports both **Snowflake Native Apps** (`application`/`application package` entity types) and **Snowflake Apps Deploy** (`snowflake-app` entity type). The entity type in `snowflake.yml` determines which flow is used automatically.

```
# run in terminal
snow app init my_native_app                          # scaffold a
Native App project
snow app setup                                       # init
snowflake.yml for App Deploy (new in 3.17.0)

snow app bundle                                      # bundle
artifacts locally
snow app deploy                                      # deploy to
Snowflake (create or update)
snow app deploy --no-prune                           # deploy
without removing extra files
snow app validate                                    # validate app
before deploying

snow app run                                         # deploy + open
in browser
snow app open                                        # open existing
app in browser

snow app events                                      # view app
event logs
snow app version create v1_0 --patch 0              # create a
release version
snow app version drop v1_0                          # drop a version

snow app teardown                                    # remove all
app objects
snow app teardown --force                            # skip
confirmation
```

Minimal `snowflake.yml` for Native App:

```yaml
definition_version: 2
entities:
  my_pkg:
    type: application package
    stage: app_stage
    manifest: manifest.yml
  my_app:
    type: application
    from:
      target: my_pkg
```

Environment Variables

Override any connection parameter without modifying `connections.toml`:

Variable	What it sets
`SNOWFLAKE_ACCOUNT`	Account identifier
`SNOWFLAKE_USER`	Username
`SNOWFLAKE_PASSWORD`	Password or PAT
`SNOWFLAKE_AUTHENTICATOR`	Auth method (e.g. `OAUTH_AUTHORIZATION_CODE`)
`SNOWFLAKE_PRIVATE_KEY_RAW`	Raw private key for key-pair auth
`SNOWFLAKE_DEFAULT_CONNECTION_NAME`	Override the default connection
`SNOWFLAKE_HOME`	Override config directory (default `~/.snowflake`)
`SNOWFLAKE_CONNECTIONS_<NAME>_<PARAM>`	Override one param of a named connection
`SNOWFLAKE_ENHANCED_EXIT_CODES`	`1` for differentiated exit codes (2=bad params, 5=query error)

Per-connection override example — override just the role for a named connection `myconn`:

```
# run in terminal
SNOWFLAKE_CONNECTIONS_MYCONN_ROLE=analyst snow sql -q "SELECT
current_role()"
```

Tips & Gotchas

- **`-x` is the CI pattern.** `snow connection test -x` and `snow sql -f script.sql -x` use environment variables (`SNOWFLAKE_ACCOUNT`, `SNOWFLAKE_USER`, etc.) instead of `connections.toml` — no config file needed on runners. Pair with OIDC for secretless CI.
- **`<% var %>` is the recommended template syntax.** The legacy `&variable_name` still works but is deprecated. Use `<% ctx.env.VAR %>` to read from environment variables passed with `--env VAR=value`, not `&{ctx.env.VAR}` (mixed syntax that does not work).
- **`--local-only` for trusted SQL enforcement.** Added in 3.19.0: prevents `!source` and `!load` from fetching URLs, so SQL files cannot pull in remote content. Use in CI when you want to guarantee no outbound requests from SQL execution.
- **`snow app` entity type is automatic.** Since 3.17.0, the same `snow app deploy` command works for both Native Apps and

Snowflake Apps Deploy — the `type:` field in `snowflake.yml` selects the behavior. No separate command groups needed.
- **Private key passphrase lookups changed in 3.17.1.** The passphrase is now read from `private_key_file_pwd` or `private_key_passphrase` in config. The `PRIVATE_KEY_PASSPHRASE` env var still takes precedence when set.

References

- Snowflake CLI documentation
- CLI command reference
- snow sql — executing SQL
- Snowflake CLI 2026 release notes
- Cortex LLM supported models